

SemSched: Semantic Sub-Layer-Aware Scheduling for Hybrid CXL-Based LLM Inference

Md Romyull Islam, Tu N. Nguyen, Selena He, Yong Shi, Kun Suo

Department of Computer Science, Kennesaw State University, GA, USA

Email: mislam22@students.kennesaw.edu, {she4, tu.nguyen, yshi5, ksuo}@kennesaw.edu

Abstract—Modern large language models (LLMs) used in long-context applications produce working sets that often exceed typical DRAM capacity, leading to memory-induced stalls during inference. Hybrid Compute Express Link (CXL) devices combine a fast device-side DRAM cache with high-capacity NAND storage on a single PCIe card, a promising platform for memory expansion. However, current software treats this as passive storage and fails to use its internal hierarchy or full-duplex links. This causes latency stalls and wastes available bandwidth. We propose SemSched, a Hybrid-CXL orchestration layer built on a single insight: LLM scheduling on hybrid DRAM-NAND memory becomes tractable when decoder blocks are split into attention and MLP sub-layers and scheduled separately. SemSched proactively places and prefetches attention and MLP blocks according to their memory behavior, stages NAND-resident layers through an asynchronous I/O pool that overlaps with prefill compute, and schedules KV-cache writes on the otherwise idle CXL write lane. On a 72B model, SemSched delivers a $1.67\times$ decode speedup at FP16 16H+64C, $1.45\times$ at INT8 16H+32C, and $1.23\times$ at FP32 32H+64C over the strongest streaming baseline, while matching or slightly exceeding demand paging on prefill. We further show that these gains are contingent on the compute engine: below roughly 3.5 TFLOPS the workload is compute-bound and the advantage disappears entirely, which both bounds the applicability of our design and clarifies why memory-hierarchy scheduling matters precisely on the fast accelerators used in practice.

Index Terms—CXL, Hybrid Memory, LLM Inference, Semantic Scheduling, KV-Cache Management, Prefetching

I. INTRODUCTION

LLMs used in long-context applications produce working sets that exceed typical server DRAM, making memory capacity and bandwidth major bottlenecks in token generation [1]. Compute Express Link (CXL) technology offers a solution, letting CPUs and GPUs access large memory pools via standard load/store instructions. Recent measurements [2]–[4] show CXL provides the bandwidth and latency needed to scale memory capacity at low cost.

Hybrid CXL devices (CMM-H) combine on-device DRAM cache with high-capacity NAND flash on a single PCIe card, forming a two-level memory hierarchy [5]. Existing software treats hybrid CXL as a single passive storage layer with reactive, page-level demand paging, failing to use the DRAM-NAND separation or the full-duplex CXL link. The result is underutilized write-channel bandwidth, increased latency

from synchronous NAND access, and cache thrashing from indiscriminate eviction.

CMM-H supports two modes: device-managed (DRAM acts as a transparent LRU cache, flat namespace) and host-managed (the CXL 2.0 CXL.mem protocol exposes device DRAM as a NUMA node accessible via load/store instructions [5]). SemSched uses host-managed mode because device-managed LRU cannot distinguish attention sub-layers from MLP sub-layers, evicting critical attention blocks at the same rate as dense MLPs under capacity pressure. The vendor characterization confirms this replacement policy is 8-way LRU with MRU insertion [5], [6], so the limitation is a property of the shipping hardware rather than an assumption.

SemSched models hybrid CXL memory as a three-layer hierarchy (host DRAM, CXL device DRAM, CXL device NAND) and exploits the predictable execution order of Transformer layers to prefetch upcoming blocks and overlap transfers with compute. Because attention and MLP sub-layers exhibit distinct memory behaviors (varying sizes, sparsity, latency sensitivity), SemSched splits blocks into semantic components and routes them across tiers accordingly. Once attention and MLP are addressable separately, semantic placement, prefill warmup, KV-write scheduling, and sparsity-aware eviction all follow as natural consequences. Unlike prior work that offloads compute [7] or pools DRAM tiers without write-channel awareness [8], SemSched is the first to unify semantic sub-layer-aware placement with full-duplex CXL scheduling in a single orchestration layer.

Scope. SemSched targets *batched serving* ($B \geq 16$) of typical chat workloads with prefill lengths below roughly 1000 tokens, running on a compute engine fast enough that the workload remains memory-bound. This regime matters because it is where hosted inference services operate: throughput per socket, not single-request latency, sets deployment cost. Outside it — single-user latency ($B=1$), very long prompts ($\geq 2K$ tokens), or slow compute engines — simpler streaming policies match or beat SemSched, and we quantify each boundary in §V. We state this up front because the applicability limits are a result of this work, not a caveat to it.

Table I summarizes our three contributions, each tied to a specific gap in current CXL inference stacks: (1) Semantic Prefill Warmup with sub-layer-aware lookahead, (2) Sparsity-Aware Placement, and (3) Opportunistic Write Scheduling on the idle CXL write lane.

TABLE I: SemSched Roadmap: Connecting Research Gaps to Contributions

#	Research Gap	Motivation Case Study	SemSched Contribution
1	I/O Latency Wall Existing stacks cause reactive NAND stalls.	72B model performance collapses due to synchronous fetches (Fig. 1a).	Phase 1: Semantic Prefill Warmup stages weights into device DRAM on an asynchronous I/O pool that overlaps with prefill compute Phase 2: Sub-layer-aware lookahead overlaps transfers with compute during decode.
2	Semantic Blindness 8-way LRU device caching evicts critical layers.	Policies hit ceiling once working set exceeds DRAM (Fig. 1b).	Pre-Execution & Phase 2: Sparsity-Aware Placement prioritizes pinning of attention and high-sparsity MLP sub-layers.
3	Bus Under-utilization Inference is read-dominant.	CXL link underused during decode (0% write utilization) (Table. V).	Phase 2: Opportunistic Write Scheduling injects KV-cache writes on the idle CXL lane without interrupting the read stream.

II. BACKGROUND AND MOTIVATION

A. The Memory Crisis and CXL

Modern LLMs need memory capacities far beyond standard server configurations. Llama 2 70B requires about 140 GB in half-precision [9], creating a capacity gap for deployment at scale. Memory cost differences amplify this gap: NAND flash is an order of magnitude cheaper (\$0.15/GB) than DRAM at \$10.00–\$12.50 per gigabyte [10], [11]. Hybrid-CXL devices address this by pairing high-speed device DRAM with large-capacity NAND on one platform. The resulting three-tier hierarchy enables memory expansion for LLM inference [5].

NVMe-SSD host-side tiering is limited by the software I/O stack: interrupts and context switches add 10–100 μ s per access. Hybrid CXL bypasses this layer and reaches device NAND in a few μ s via CXL.mem [6].

B. Transformer Memory Characteristics

Autoregressive models process tokens through a fixed pipeline. Layers run in a deterministic order with known sizes, making memory access patterns predictable and allowing efficient prefetching [1], [7].

Operational Heterogeneity. Each decoder block contains a self-attention layer and an MLP [12], with distinct resource demands. Self-attention is memory-bound and latency-sensitive; MLP has higher arithmetic intensity. We exploit this diversity to mask memory latency by overlapping compute-heavy MLP execution with asynchronous weight transfers.

Quantization. Modern quantization reduces model size by 2–8 $\times$ while speeding up compute [13]. Faster compute leaves less time to overlap data transfers, so the processor often waits idle for the next layer. CXL bandwidth is lower than host DRAM, so the interface becomes a persistent bottleneck unless a hierarchy-aware caching layer masks link latency [7].

C. Motivation and Roadmap

Standard software stacks fail to use Hybrid-CXL effectively. Table I summarizes the gap between standard schedulers and the hardware hierarchy.

The Performance Collapse. Scaling from 13B to 72B parameters causes a sharp performance drop across standard baselines: capacity-oriented (FlexGen), OS-integrated (LIA), and sparsity-aware (LLMFlash). The 72B model collapses under NAND-bound configurations, with the system spending the overwhelming majority of decode time stalled on reactive NAND fetches [5]. SemSched addresses this by staging weights before decoding begins.

The Semantic Gap. Reactive caching (LRU, LFU) incurs a substantially higher miss penalty than pinning policies. As shown in Fig. 1(b), LRU and LFU perform identically: the periodic, sequential nature of LLM inference gives every layer equal frequency and recency priority, causing constant cache thrashing. We define the *cumulative per-token miss penalty* as the additional time a layer costs when served from NAND rather than from device DRAM, summed over all layers in one decode step:

$$\Pi = \sum_{\ell} \max(0, \max(T_{\text{comp}}^{\ell}, T_{\text{NAND}}^{\ell}) - \max(T_{\text{comp}}^{\ell}, T_{\text{CXL}}^{\ell})).$$

Static and semantic pinning look equivalent when DRAM holds the critical layers, but under capacity pressure static tiering fails to adapt to dynamic sparsity. SemSched breaks this ceiling by using semantic sub-layer awareness to selectively pin components.

The Bandwidth Underutilization. Standard inference is read-dominant [4]. Modern memory controllers use write buffers and favor reads, but reads and writes share one bus and must take turns. The vendor characterization of CMM-H makes this explicit, noting that back-to-back read issue “cannot make use of the full-duplex PCIe link” [6]. When LLMs spill parameters and KV to Hybrid-CXL, writes compete with reads on the same link, and switching between them creates gaps where the CXL lane carries little useful read traffic. SemSched treats the CXL device transmit path as a separate write lane and schedules KV-cache writes there, so the receive path can focus on fetching weights and KV.

III. RELATED WORK

CXL Systems and Memory Expansion. Foundational studies [14]–[16] characterize CXL bandwidth and latency. For LLMs, recent work focuses on offloading the Key-Value (KV) cache. Tang et al. [1] show CXL-based KV placement enables 30% larger batch sizes and reduces GPU requirements by up to 87%. Beluga [17] adds direct GPU access to shared memory via a CXL switch, improving throughput by 7.35 $\times$ over RDMA. OASIS [18] clusters outlier data to address CXL bandwidth bottlenecks. Liaw and Chen [19] target memory allocation for fine-tuning with a 1.8 $\times$ speedup. Tang et al. [20] extend CXL to cluster-scale training. These systems use CXL for *state* or *training*; we focus on *inference parameters*. Unlike generic pooling [3], SemSched exploits the DRAM/NAND hierarchy of Hybrid-CXL devices and shapes traffic to use full-duplex bandwidth [4]. **[TODO: refresh with 2025–26 CXL-for-LLM work: near-data processing for CXL bandwidth]**

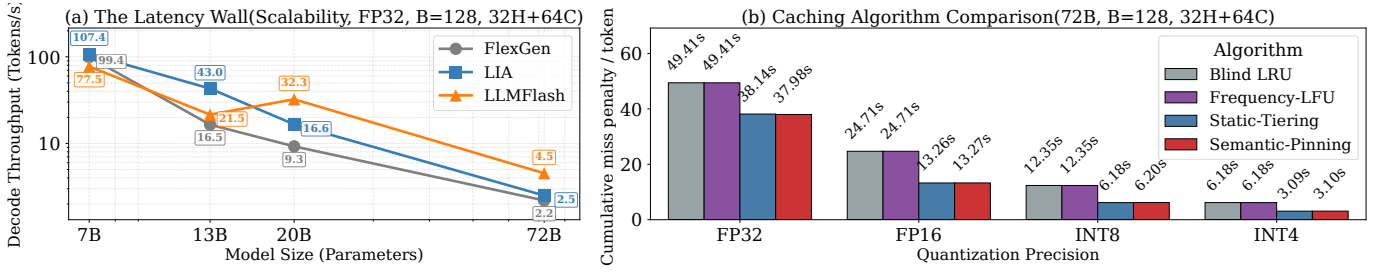


Fig. 1: (a) Decode throughput scaling reveals the latency wall. (b) Cumulative per-token miss penalty across caching policies. [TODO: regenerate from corrected simulator]

TABLE II: System Configuration Parameters. Memory configurations use the shorthand $\langle \text{host} \rangle \text{H} + \langle \text{cxl} \rangle \text{C}$, e.g. 16H+32C denotes 16 GB host DRAM plus 32 GB CXL device DRAM.

Component	Specification	Source
Compute	Xeon 6 (AMX), 64C @ 2.0 GHz, 1024 FLOP/cyc/core BF16, 88% eff. = 115.3 TFLOPS sustained	vendor spec
Host DRAM	16–32 GB, DDR5-4800, 38.4 GB/s, 200 ns	JEDEC
CXL DRAM	32–64 GB, 27 GB/s, 505 ns	[5]
CXL NAND	256 GB, 5 GB/s, 1.5 μs	[5]
NVMe SSD	512 GB, 7.6 GB/s (Gen4), 20 μs	Gen4 x4
Models	Mistral 7B, Llama 13B, Qwen3 20B, Qwen2.5 72B	
I/O chunk	256 KB (NAND/SSD access granularity)	
Quantization	FP32 (4B), FP16 (2B), INT8 (1B), INT4 (0.5B)	
Seq. length	512 tokens (prefill/KV)	
Decode tokens	16 (gen phase)	
Batch size	1–128	

amplification; CXL-enabled KV-cache management beyond GPU limits; TraCT rack-scale disaggregated serving; Equilibria multi-tenant CXL tiering]

Hardware-Software Co-Design and Tiering. Hardware-software co-design has been used to overcome physical limits. CENT [2] uses GDDR6-PIM for $2.3\times$ higher throughput than GPU baselines. Park et al. [21] propose LPDDR-based CXL-PNM, reducing TCO by 40%. Liu et al. [22] report $2.1\times$ gains for recommendation models via near-CXL processing. SkyByte [23] bridges byte-addressable CXL and page-addressable flash through an OS-co-designed controller. On the software side, Transparent Page Placement (TPP) [24] promotes pages by access frequency, and HeMem [25] improves NVM scalability via asynchronous batching. These approaches are reactive and incur hotness-detection latency. SemSched instead leverages deterministic LLM decoding for proactive, zero-stall data movement without OS-level context switches.

Inference Offloading and Optimization. Before CXL, software systems addressed the memory wall via offloading. DeepSpeed-Inference [26] and FlexGen [27] pioneered aggressive CPU/NVMe offloading for massive models on a single GPU. They optimize throughput via zig-zag scheduling but treat storage as a block device (10–100 μs); large batch sizes degrade interactive performance. The closest work to ours is LIA [7], which uses CXL for single-GPU inference but offloads *computation* to the CPU via AMX instructions. SemSched instead hides latency within the memory hierarchy using the CXL device’s internal DRAM cache. Standard optimizations complement our approach: quantization (e.g., W4A8KV4 [28]) reduces memory footprints by 1.2–3.5 $\times$ but

accelerates compute faster than the CXL link can supply data, making tier-aware placement more critical. Continuous batching (Orca [29], vLLM [30]) maximizes iteration-level concurrency, and memory-aware scheduling [31] improves throughput by 8–28%. Sparsity-prediction systems (Deja Vu [32], layer-dependent activation sparsity [33]) skip computation by predicting zero-activations. SemSched repurposes these insights for memory management, using the *effect* of sparsity on working set size to prioritize memory-bound layers for the CXL DRAM cache; it does *not* skip compute.

IV. SYSTEM DESIGN

A. System Assumptions and Parameters

Our simulator models commodity hardware bandwidths and memory hierarchies [5], [15]. Current CMM-H devices fill their DRAM cache on demand with no application-aware prefetching, so a software runtime is needed to stage layers ahead of use. Existing CXL simulators such as CXLMemSim [34] and CXL-SSD-Sim [35] do not model the CMM-H DRAM-over-NAND cache at sub-layer granularity, so we built a trace-driven Python simulator for layer-by-layer inference. Hardware, workload, and quantization parameters appear in Table II.

Device bandwidths. The CXL tier parameters follow the CMM-H prototype of Soltaniyeh et al. [5]: a PCIe Gen5 $\times 8$ device with a 48 GB DDR4 cache, measured at ≈ 27 GB/s on DRAM-cache hits and ≈ 5 GB/s once the working set exceeds the cache. We note that a separate characterization [6] reports 4.5 GB/s for a different, earlier prototype with a PCIe Gen4 $\times 8$ host link and a 16 GB cache; the two figures describe different devices, and we model the Gen5 part throughout.

Compute engine. We model an AMX-capable Intel Xeon 6 sustaining 115.3 TFLOPS. This choice is load-bearing rather than incidental. Decode arithmetic intensity at batch B is $2B/\text{bytes-per-param}$, so at $B=128$ in FP16 it is 128 FLOP/byte, and saturating the 27 GB/s CXL DRAM tier requires at least 3.46 TFLOPS. Below that threshold the engine, not the memory hierarchy, sets the step time, and tier placement becomes irrelevant — a regime we quantify in §V-E. Modeling a slow CPU would therefore not make our mechanism look better; it would erase it. We use the same compute model for all four simulators, since compute

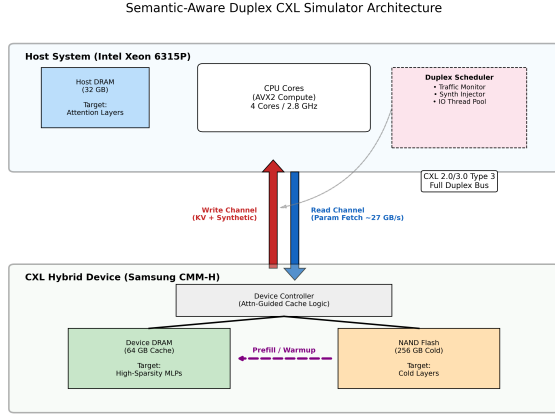


Fig. 2: SemSched orchestrates the placement of model layers across Host DRAM, CXL Device DRAM, and NAND based on semantic importance.

is a property of the simulated hardware rather than of any baseline’s policy.

Workload Model. We simulate autoregressive inference with batch sizes and token prefill from Table II, followed by a 16-token decode per batch. Each decode token triggers all model layers sequentially, with incremental KV-cache growth per batch slot.

Compute Model. Layer execution time is $T_{\text{comp}}(\ell) = \text{FLOPs}(\ell) / f_{\text{engine}}$, with $\text{FLOPs}(\ell) = 2P_{\ell}B$ during decode and $2P_{\ell}SB$ during prefill for sequence length S . FLOPs scale with batch size in every simulator.

Transfer Latency Model. Layer transfer latency is $T_{\text{transfer}}(\ell) = \text{bytes}(\ell) / \text{BW}_{\text{tier}} + L_{\text{tier}}$, following the parametric methodology of [5].

Layer Representation & Modeling. Simulated parameters remain dense, and no simulator receives a sparsity-based FLOP discount except LLMFlash, for which sparsity-driven compute skipping is the mechanism its own paper contributes. FlexGen and LIA treat decoder blocks as monolithic units for reactive paging. LLM-in-a-Flash decomposes blocks into attention and MLP sub-layers and pools Host and CXL DRAM into a single tier. SemSched also decomposes blocks but routes them semantically: prioritizing attention and high-sparsity MLPs for Host DRAM, while denser MLPs are mapped to CXL Device DRAM.

B. Baseline Simulators

We selected three baselines spanning the design space of LLM offloading and CXL memory management: FlexGen for capacity-oriented offloading, LIA for OS-integrated CXL tiering, and LLM-in-a-flash for hardware-aware asynchronous scheduling.

Baseline fidelity principle. Each baseline follows its own paper’s *policy* — placement, eviction, streaming strategy, and published constants — while all four share identical *hardware* modeling: compute rate, FLOP accounting, tier bandwidths and latencies, KV growth, and metric definitions. A baseline may be advantaged or disadvantaged only by its own design

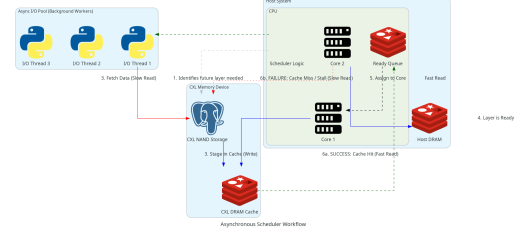


Fig. 3: The SemSched Scheduler monitors the CXL read/write ratio in real-time and controls semantic-aware placement.

decisions, never by our modeling of the machine. Any deviation from a source paper is stated explicitly below.

Capacity-Optimized Simulator: FlexGen. We implement the FlexGen [27] memory policy. FlexGen uses a “Strict Block-Diagonal Offloading” strategy, treating memory as a unified hierarchy (GPU → CPU → Disk). Our simulator mimics this by pinning hot layers (embeddings, heads) to Host DRAM and streaming the remaining decoder blocks from SSD (or CXL NAND) via demand paging. This maximizes capacity but suffers from serialization latency.

System-Optimized Simulator: LIA. We implement LIA’s [7] memory policy. LIA places all model parameters in CXL device DRAM, reserving Host DRAM exclusively for KV-cache writes and activations. Overflow spills to CXL Device NAND. Weight reads use the CXL bus and KV writebacks the DDR bus; since these are physically separate, we model them as overlapping with compute. A conservative serial-sum variant gives results within 0.3%. *Deviation:* we omit LIA’s cooperative AMX CPU–GPU compute offloading and model only its CXL memory tiering. This is defensible at our operating point: at $B=128$, LIA’s own optimal policy assigns all parameter-dependent sublayers to the accelerator, and our shared compute engine already sits far above the 3.46 TFLOPS memory-bound threshold, so compute placement is not decisive for throughput.

Sparsity Optimized Simulator: LLMFlash. We implement a CXL-adapted baseline based on LLM-in-a-flash [8]. LLMFlash pools Host and CXL DRAM into a unified tier — modeled here as a capacity-weighted blend of the two DRAM bandwidths — and uses “selective persistence” to pin attention weights in DRAM. For MLP sub-layers it exploits dynamic activation sparsity, streaming only active neurons from NAND via a sliding window ($k=5$) with row-column bundling ($1.8 \times$ NAND throughput). Because sparsity-driven compute skipping is this paper’s own contribution, LLMFlash is the one baseline that receives a sparsity FLOP discount. Its structural limits are inherited from the design: during prefill all FFN neurons must fire, forcing a full FFN reload from CXL NAND per token; during decode, the union of active neurons across sequences causes “sparsity collapse” once the expanded working set overflows the DRAM window.

C. The SemSched Architecture

The SemSched framework implements the semantic placement strategy in Figure 2. We use the term “semantic” because the scheduler leverages the functional properties of Transformer sub-layers (Attention vs. MLP) to inform its decisions. The system runs a pre-execution static analysis followed by two runtime phases.

Pre-Execution: Static Analysis and Placement. The Static Analyzer computes per-layer sparsity to establish Semantic-Aware Placement. Decoder blocks contain comparable parameter counts, but activation sparsity varies across the model architecture [33]. Placement proceeds by priority: output head, then latency-sensitive Attention sublayers, then embeddings, then MLP sublayers whose sparsity exceeds 0.60, all into Host DRAM while capacity remains (**Gap 2**). Remaining low-sparsity, compute-bound MLP sublayers are assigned to CXL device DRAM, where their higher arithmetic intensity masks the added CXL latency. Sparsity here is a static, architectural property derived from published per-layer profiles [33]; it takes no workload input, so placement is fixed for a given model and memory configuration and does not vary with the request stream. Real systems use standard OS tools (`mmap`, `mbind`) to lock high-priority memory into the CXL device as a zero-core virtual NUMA node. Layers assigned to Host DRAM remain pinned for the entire session.

Phase 1: Asynchronous Parallel Warmup (Prefill). Before the first decode token is issued, SemSched stages all NAND-resident layers into CXL Device DRAM using an asynchronous I/O thread pool ($|T|=16$) (**Gap 1**). Two properties matter. First, aggregate NAND bandwidth is a hard physical floor: a thread pool amortizes the per-chunk access latency but cannot multiply the device’s byte-movement rate, so layer i completes staging at $t_{\text{stage}}(i) = \sum_{j \leq i} (\text{bytes}_j / \text{BW}_{\text{NAND}} + \text{chunks}_j \cdot L_{\text{NAND}} / |T|)$. Second, staging occupies the NAND-to-device-DRAM DMA path, a resource independent of the compute engine, so it *overlaps* with prefill computation rather than preceding it: a layer stalls only if its staging has not completed when execution reaches it. Staged layers are session-pinned, preventing eviction during decode. Because per-layer prefill compute exceeds per-layer staging time at our operating point, staging is almost entirely hidden — the mechanism removes NAND fetches from the decode critical path at close to zero prefill cost, rather than by making staging free.

Phase 2: Semantic-Scheduled Decode. The controller manages the hierarchy through four components. ① **Bandwidth Monitor.** Standard inference is read-heavy ($> 99.9\%$), so the write channel stays idle [4] (**Gap 3**). The monitor tracks the CXL read/write ratio to identify unused bandwidth. ② **Write Scheduler.** Guided by the monitor, it performs Opportunistic Write Scheduling on the idle write lane, evicting cold layers using sparsity-aware scoring and streaming KV deltas to the Tx queue. Write volume per token is bounded by KV-cache growth, ensuring no contention with the dominant read stream. ③ **Attention Guided Cache.** This manages CXL Device DRAM as a dynamic cache exclusively for NAND-

TABLE III: Decode throughput (tokens/s), Qwen 72B, $B=128$, decode=16.

Configuration	FlexGen	LIA	LLMFlash	SemSched
FP16 16H+64C	4.39	6.48	11.73	19.58 (1.67 $\times$)
INT8 16H+32C	9.91	12.97	27.79	40.43 (1.45 $\times$)
FP32 32H+64C	2.20	2.51	4.64	5.70 (1.23 $\times$)

bound layers, never migrating data to or from Host DRAM. Layers are scored by $\text{Score}(j) = \text{Type}(j) \cdot \text{Sparsity}_j$, with attention weighted above MLP; high-score layers are pinned and dense layers stream from NAND [8], [33]. Note that this runtime score is distinct from the static placement rule above: placement uses a priority ordering with a sparsity threshold, whereas prefetch ranking and eviction share the multiplicative score. ④ **Execution Pipeline.** This implements Algorithm 1, using a deep lookahead window ($K=32$) to re-stage layers evicted under capacity pressure. It employs a Semantic Streaming Overlap factor ($\sigma=0.85$), letting computation start before full I/O completion. At small batch ($B < B_{th}=4$) the pipeline scales MLP transfer bytes by $1 - (1 - \text{sparsity}_i)^B$. The CXL link is modeled as two independent lanes (Rx for reads, Tx for KV writes) at 27 GB/s per lane [5], [15], [16]; the Rx lane is an accounting lane, with read timing set by the tier model. A KV write completing within the active read window incurs no penalty; otherwise its slip is charged as a stall. Tx-lane utilization is measured from queue state rather than byte accounting.

V. EVALUATION

A. Experimental Setup

Table II lists the hardware and sweep ranges. We use Qwen 72B (≈ 288 GB FP32 footprint) as the stress workload across four memory pairings {16H+32C, 16H+64C, 32H+32C, 32H+64C}. The canonical operating point is $B=128$, prefill=512, decode=16 unless stated otherwise. Reported metrics are decode throughput (TPS), prefill throughput, time-per-output-token (TPOT), per-token write stall, and SemSched-to-baseline speedup. All four simulators report decode throughput as the mean across decode steps and prefill throughput on a per-sequence basis, so figures are directly comparable.

B. End-to-End Decode Performance

Table III gives the headline result. SemSched improves decode throughput by 1.67 $\times$ at FP16 16H+64C, 1.45 $\times$ at INT8 16H+32C, and 1.23 $\times$ at FP32 32H+64C over LLMFlash, the strongest baseline. The advantage is largest where DRAM is scarcest relative to the working set, which is the regime the design targets.

For smaller models (7B, 13B) the model fits in Host and CXL DRAM; there, FlexGen and LIA match or slightly beat SemSched, since splitting layers into attention and MLP adds overhead that only pays off when NAND becomes the bottleneck. As batch sizes scale (Fig. 5), baselines suffer sparsity collapse where the union of active neurons overflows the DRAM cache. LLMFlash benefits from activation sparsity at $B=1$ but is hit at $B \geq 4$. At INT4 the working set is small

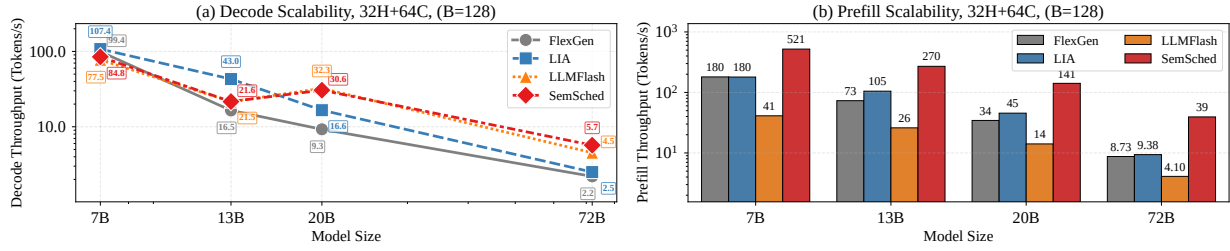


Fig. 4: (a) Decode throughput scalability. (b) Prefill throughput. [TODO: regenerate]

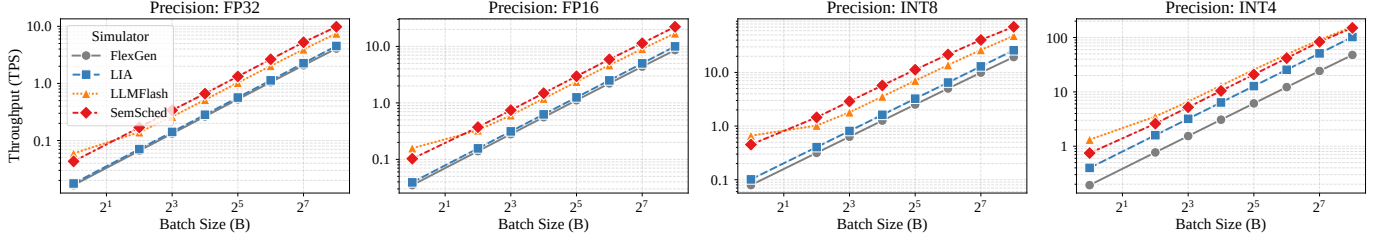


Fig. 5: Throughput scaling across varying batch sizes (16GB Host + 32GB CXL). [TODO: regenerate]

enough that LLMFlash’s sliding window remains dominant across all batch sizes.

C. Prefill Performance

At $B=128$ with a 512-token prompt, prefill requires $2PSB \approx 9.44 \times 10^{15}$ FLOPs, or 81.8 s of pure computation on our engine — for every scheduler. Prefill is therefore compute-bound, and memory scheduling cannot substantially change it: FlexGen measures 83.3 s and LIA 83.7 s, both essentially at the compute bound.

SemSched reaches 6.53 t/s against FlexGen’s 6.15 t/s at FP16 16H+64C (6.19 vs 6.16 at INT8 16H+32C; 6.09 vs 6.14 at FP32 32H+64C). The contribution of Phase 1 in this phase is therefore not acceleration but *cost concealment*: staging the entire NAND-resident working set costs 13.1 s of DMA at FP16 16H+64C, yet because it runs on a resource independent of the compute engine and per-layer compute exceeds per-layer staging, it is almost entirely hidden. SemSched removes NAND fetches from the decode critical path while paying essentially nothing in prefill. LLMFlash is a distant outlier at 0.10 t/s, a direct consequence of its design: prefill fires all FFN neurons, forcing a full ≈ 113 GB FFN reload from NAND per token.

D. Impact of Quantization and Memory Configuration

SemSched maintains a speedup over LLMFlash across precisions, growing as memory tightens. Sensitivity analysis shows SemSched’s utility depends on DRAM scarcity: under FP32 the 72B footprint creates zero-sum competition for memory and SemSched holds a consistent advantage across all four memory configurations, whereas at INT4 the working set fits in the CXL device cache and simpler streaming wins. [TODO: regenerate Figs. 6, 7 and restate exact precision speedups from the corrected sweep]

TABLE IV: Compute-engine sensitivity (Qwen 72B FP16, 16H+64C, $B=128$). The same compute model is applied to every simulator.

Engine	TFLOPS	LLMFlash	SemSched	Speedup
4-core AVX2 CPU	0.08	0.54	0.54	1.00×
16-core AVX2 CPU	0.32	2.16	2.15	1.00×
64-core AVX2 CPU	1.08	7.39	7.32	0.99×
Xeon 6 (AMX)	115.3	11.73	19.58	1.67×

E. Compute-Engine Sensitivity

A natural concern with any memory-scheduling result is that a deliberately slow compute engine manufactures an overlap window that flatters the scheduler. We test this directly by sweeping the compute rate across four orders of magnitude, applying the identical knob to all four simulators (Table IV).

The effect runs opposite to the concern. A slow engine does not inflate the advantage; it *eliminates* it. At 0.08 TFLOPS the speedup is exactly 1.00×, because computation swamps the tier difference that SemSched exists to exploit — moving a layer from NAND to device DRAM saves nothing when the processor is the bottleneck either way. The advantage appears only once the engine clears the 3.46 TFLOPS memory-bound threshold derived in §IV-A. This bounds our claim precisely: SemSched is a scheduler for systems whose compute is fast enough that memory placement determines throughput, which is the defining characteristic of modern accelerator-based serving.

F. Simulator Fidelity

Our evaluation rests on three separately grounded components. Memory-tier bandwidths and latencies are taken from published measurements of physical CMM-H hardware [5], not estimated. The two-queue CXL link model is covered by a unit-test suite of ten tests, including a hand-calculable ten-layer loop and a paper-scale 72B estimate, which verify read/write concurrency, deadline slip accounting, and queue-measured utilization. We additionally assert bandwidth conservation: no tier may exceed its physical rate over any

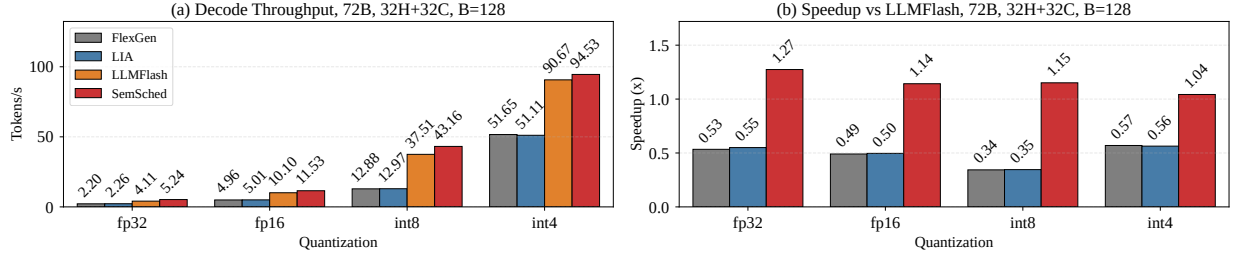


Fig. 6: (a) Decode throughput across precisions (b) SemSched speedup vs. LLMFlash at $B=128$. [TODO: regenerate]

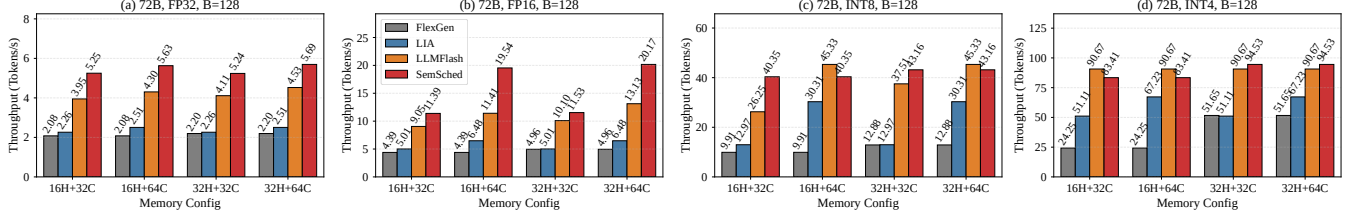


Fig. 7: Throughput sensitivity across varying memory configurations. [TODO: regenerate]

window — an invariant that makes an entire class of modeling error unrepresentable rather than merely absent. [TODO: add LLMCompass calibration of the compute term (ISCA 2024, 4.1% reported error) to close the loop on compute fidelity]

G. Mechanism Analysis and Ablation

Semantic placement keeps attention layers in Host DRAM where bandwidth pressure is highest, and pushes MLP weights to CXL DRAM where higher arithmetic intensity hides the added transfer time. The duplex link injects writes during active read cycles, bypassing the sequential bus-turnaround penalty; the queue-measured Tx-lane utilization stays small because KV writes are low in volume yet cause large stalls when serialized. [TODO: re-run trace_workload/ablation.py and restate Table VI, the write-stall reduction, and Table V TPOT figures from the corrected simulator]

H. Decode Latency Model

A simple analytical model explains the measured gains. Per-step decode latency is $T_{\text{step}} = \max(T_{\text{comp}}, T_{\text{weight}}) + T_{\text{KV-read}} + T_{\text{KV-stall}}$, where T_{weight} depends on the layer's tier, $T_{\text{KV-read}} = (P + t) \cdot k \cdot B / \text{BW}_{\text{KV}}$ grows linearly with prefill length P and decode step t , and $T_{\text{KV-stall}} = \max(0, T_{\text{KV-write}} - T_{\text{read-window}})$ is the KV write time left exposed after the duplex link hides it inside the read-occupancy window. Phase 1 warmup forces T_{weight} to use $\text{BW}_{\text{CXL-DRAM}}$ instead of BW_{NAND} , so the SemSched/LLMFlash speedup is approximately

$$S(P) \approx \frac{T_{\text{comp}} + T_{\text{weight}}^{\text{NAND}} + T_{\text{KV-read}}(P)}{T_{\text{comp}} + T_{\text{weight}}^{\text{CXL}} + T_{\text{KV-read}}(P)}.$$

The model makes two predictions borne out by the measurements. At short P the NAND term dominates and $S(P)$ is large; at long P , $T_{\text{KV-read}}$ dominates both numerator and denominator and $S(P) \rightarrow 1$, giving a prefill-dependent crossover. It also explains §V-E: as T_{comp} grows it appears

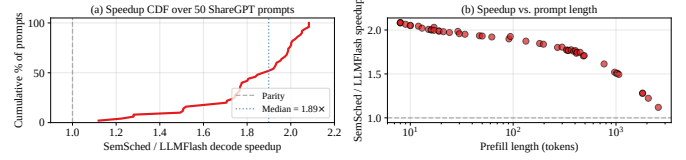


Fig. 8: ShareGPT validation at FP16 16H+64C, $B=128$, decode=64. (a) CDF of SemSched/LLMFlash decode speedup across 50 real prompts. (b) Speedup vs. prompt length. [TODO: regenerate]

in both terms, driving $S \rightarrow 1$ regardless of tier placement, which is precisely the 1.00 $\times$ observed on slow engines.

I. Real-Workload Validation on ShareGPT

Our simulator reads only the sequence-length distribution from ShareGPT, not the prompt text. Per-layer compute, transfer, and KV-cache costs come from the Qwen2.5 72B architecture combined with bytes-and-bandwidth movement under each placement policy; the simulator never executes attention or matrix multiplication. This matches standard methodology for memory-system simulators, including the CMM-H characterization of [5]. We replay 50 randomly sampled prompts from the ShareGPT V3 corpus at FP16 16H+64C, $B=128$, decode=64 tokens. The sampled prefill distribution has median 114 tokens, P90 1038, and max 2579, covering typical chat workloads. [TODO: re-run trace_workload/trace_runner.py on the corrected simulator and restate the median speedup, win-rate, P10/P90, and Fig. 8]

VI. DISCUSSION

SemSched is most effective when the working set exceeds DRAM capacity, the workload tolerates batching, and the compute engine is fast enough to leave the system memory-bound. Under those conditions it enables 72B deployment on commodity sockets with standard DDR and NAND hierarchies rather than HBM clusters. When the working set fits entirely

TABLE V: Decode Hardware Efficiency and Speedup (Qwen 72B FP16, $B=128$, 16H+64C).

Scheduler	Placement	TPOT (s/tok)	U_{write} (%)	Speedup
FlexGen	Monolithic Paging	0.228	0.00%	$0.37\times$
LIA	NUMA Tiering	0.154	0.00%	$0.55\times$
LLMFlash	Unified Streaming	0.085	0.00%	$1.00\times$ (Ref)
SemSched	Semantic Pinning	0.051	0.03%*	$1.67\times$

* U_{write} is the queue-measured Tx-lane utilization.
Speedup = $TPOT_{Ref}/TPOT_{SemSched}$; decode-phase only.

Algorithm 1 SemSched: Unified Staging and Execution

Require: Sub-layers L ; Placement P ; Cache C ; CXL Link Q ; I/O Threads $|T| = 16$; Lookahead $K = 32$; Overlap $\sigma = 0.85$; Adaptive Batch Threshold $B_{th} = 4$

- 1: $t_{stage} \leftarrow 0$; $stage_finish \leftarrow \emptyset$
- Phase 1: Asynchronous Parallel Warmup (Prefill)**
- 2: **for** each layer $i \in L$ where $P[i] = \text{NAND}$ **do**
- 3: $t_{bw} \leftarrow L[i].bytes / BW_{NAND}$ $\triangleright$ bandwidth is a hard floor
- 4: $t_{lat} \leftarrow \text{chunks}(L[i]) \cdot \mathcal{L}_{NAND} / |T|$ $\triangleright$ threads amortize latency only
- 5: $t_{stage} \leftarrow t_{stage} + t_{bw} + t_{lat}$
- 6: $stage_finish[i] \leftarrow t_{stage}$
- 7: $C.add(i, L[i].bytes)$; $C.session_pin(i)$
- 8: **end for**
- Prefill execution (staging overlaps with compute)**
- 9: $lat \leftarrow 0$
- 10: **for** each layer $i = 1$ to $|L|$ **do**
- 11: $t_{comp} \leftarrow \text{compute_time}(2 \cdot P_i \cdot S \cdot B)$
- 12: $start \leftarrow \max(lat, stage_finish[i])$ $\triangleright$ stall only if not staged
- 13: $lat \leftarrow start + \text{pipe}(t_{comp}, \text{transfer_time}(i, P[i]))$
- 14: **end for**
- Phase 2: Semantic-Scheduled Decode**
- 15: **for** token $t = 1$ to N_{gen} **do**
- 16: $w \leftarrow 0$
- 17: **for** each layer $i = 1$ to $|L|$ **do**
- 18: $t_{comp} \leftarrow \text{compute_time}(2 \cdot P_i \cdot B)$ $\triangleright$ dense; scales with batch
- 19: **if** $L[i]$ is MLP **and** $B < B_{th}$ **then**
- 20: $L[i].bytes \leftarrow L[i].bytes \times (1 - (1 - \text{sparsity}_i)^B)$
- 21: **end if**
- 22: $S \leftarrow \{j \in [i+1, \dots, i+K] \mid P[j] = \text{NAND}, j \notin \text{fetched}\}$
- 23: Rank S by $Score = \text{Type}(j) \cdot \text{Sparsity}_j$ and assign to idle T
- 24: **if** $C.contains(i)$ **then**
- 25: $step_lat \leftarrow \max(t_{comp}, \text{transfer_time}(i, \text{CXL-DRAM}))$
- 26: **else**
- 27: $t_{stall} \leftarrow \max(0, \text{Ready}(i, T) - w)$
- 28: $step_lat \leftarrow t_{stall}(1 - \sigma) + \max(t_{comp}, \text{transfer_time}(i, \text{NAND}))$
- 29: **end if**
- 30: **if** $L[i]$ is Attention **then**
- 31: $kv_read \leftarrow (S + t) \cdot kv_per_tok \cdot B$
- 32: $step_lat \leftarrow step_lat + \text{transfer_time}(kv_read, \text{CXL-DRAM})$
- 33: **end if**
- 34: **if** $\text{has_kv}(i)$ **and** $P[i] \in \{\text{CXL-DRAM}, \text{CXL-NAND}\}$ **then**
- 35: $exposed \leftarrow Q.schedule_write(t_{now}, kv_bytes, t_{now} + step_lat)$
- 36: $step_lat \leftarrow step_lat + exposed$
- 37: **end if**
- 38: $w \leftarrow w + step_lat$; $C.evict_by_sparsity_score()$
- 39: **end for**
- 40: $W_t \leftarrow w$ $\triangleright$ throughput uses the mean over all N_{gen} steps
- 41: **end for**

within the CXL device’s DRAM (e.g., INT4), simpler unified streaming achieves higher throughput. Because placement is determined upfront and eviction runs asynchronously, the scheduler does not slow down the critical path.

Limitations. Three boundaries deserve emphasis. First, the gains are conditional on compute: below roughly 3.5 TFLOPS the workload is compute-bound and SemSched provides no benefit at all (§V-E). Second, we model the CXL-to-host memory path; a GPU-attached deployment adds a PCIe hop from host to device memory that we do not model end-to-

TABLE VI: Per-mechanism ablation on Qwen 72B ($B=128$, prefill=512, decode=16). **[TODO: re-run]**

Variant	FP16 16H+64C		INT8 16H+32C	
	TPS	W-Stall (ms)	TPS	W-Stall (ms)
LLMFlash (Ref)	11.73	—	27.79	—
Full SemSched	19.58	—	40.43	—
— Parallel Warmup	—	—	—	—
— Semantic Placement	—	—	—	—
— Two-Queue Link	—	—	—	—
— Sparsity Eviction	—	—	—	—

end, though the ordering of tier bandwidths — and hence the placement decisions — is preserved, since even a Gen5 $\times 16$ GPU link is faster than the CXL device it would be fed from. Third, prefill is compute-bound at our operating point, so the prefill contribution is cost concealment rather than acceleration.

CMM-H devices provide no device-side hardware prefetcher, and even if present, a hardware prefetcher has no knowledge of layer boundaries or the large transfer sizes CXL inference requires [5]; a software runtime is therefore the only way to stage layers proactively. Our evaluation uses a trace-driven simulator based on documented CMM-H specifications; while accurate for design conditions, it does not capture PCIe jitter, operating-system noise, or the device DRAM cache’s 8-way conflict misses, which prior measurements show can expose tail latencies once the footprint exceeds a fraction of the cache [6]. That effect penalizes device-managed caching and not explicit placement, so modeling it would widen rather than narrow our reported gap. We assume uniform decoder block sizes and use published sparsity profiles [33] rather than per-model measurements. Physical CMM-H validation is the primary focus of future work.

VII. CONCLUSION

We introduced SemSched, a scheduler that co-designs memory placement with the semantic properties of Transformer layers. By distinguishing memory-bound Attention from compute-bound, sparse MLP sub-layers, SemSched enables proactive, semantic-aware data staging. On a 72B model it delivers $1.23\times$ to $1.67\times$ decode-throughput improvement over the strongest baseline while paying essentially nothing in prefill, because staging runs on a DMA path independent of computation. We further show the result is conditional in an instructive way: the advantage vanishes entirely on slow compute engines and appears only once the system is genuinely memory-bound. Semantic-aware orchestration is therefore most valuable ex-

actly where inference is heading — fast accelerators paired with deep, cheap memory hierarchies.

REFERENCES

- [1] Y. Tang *et al.*, “Exploring CXL-based KV cache storage for LLM serving,” in *NeurIPS Workshop on Machine Learning for Systems*, 2024.
- [2] Y. Gu *et al.*, “PIM is all you need: A CXL-enabled GPU-free system for large language model inference,” in *Proc. ASPLOS*, 2025, pp. 862–881.
- [3] H. Li *et al.*, “Pond: CXL-based memory pooling systems for cloud platforms,” in *Proc. ASPLOS*, 2023, pp. 574–587.
- [4] Y. Yang *et al.*, “CXLaimPod: CXL memory is all you need in AI era,” 2025.
- [5] M. Soltaniyeh, G. Sun, X. Yao, A. Beygi, R. Kachare, D. Zhao, H. Huen, A. Chang, S. Murugesapandian, and C. Kahn, “Revisiting memory hierarchies with CMM-H: Use device-side caching to integrate DRAM and SSD for a hybrid CXL memory,” in *Proc. 17th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2025, pp. 45–51.
- [6] J. Zeng, S. Pei, D. Zhang, Y. Zhou, A. Beygi, X. Yao, R. Kachare, T. Zhang, Z. Li, M. Nguyen, R. Pitchumani, Y. S. Ki, and C. Jung, “Performance characterizations and usage guidelines of Samsung CXL memory module hybrid prototype,” 2025.
- [7] H. Kim *et al.*, “LIA: A single-GPU LLM inference acceleration with co-operative AMX-enabled CPU-GPU computation and CXL offloading,” in *Proc. International Symposium on Computer Architecture (ISCA)*, 2025, pp. 544–558.
- [8] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, “LLM in a flash: Efficient large language model inference with limited memory,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [9] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” 2023.
- [10] C. Cadenas, “RAM prices are wild in 2026, yet good PC deals still exist,” 2026, <https://tinyurl.com/4nfsmf6w>.
- [11] M. Kan, “As DRAM costs soar, prices for NVMe SSDs start to rise too,” 2025, <https://tinyurl.com/5d5fr73h>.
- [12] A. Q. Jiang *et al.*, “Mistral 7B,” 2023.
- [13] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “GPT3.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [14] X. Wang *et al.*, “Exploring and evaluating real-world CXL: Use cases and system adoption,” 2024.
- [15] —, “Performance characterization of CXL memory and its use cases,” in *Proc. IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025, pp. 1048–1061.
- [16] J. Liu *et al.*, “Dissecting CXL memory performance at scale: Analysis, modeling, and optimization,” 2024.
- [17] X. Yang *et al.*, “Beluga: A CXL-based memory architecture for scalable and efficient LLM KV cache management,” 2025.
- [18] M. Seo *et al.*, “OASIS: Outlier-aware KV cache clustering for scaling LLM inference in CXL memory systems,” *IEEE Computer Architecture Letters*, 2025.
- [19] Y. Liaw and S. Chen, “Analysis and optimized CXL-attached memory allocation for long-context LLM fine-tuning,” 2025.
- [20] Y. Tang *et al.*, “Simulating LLM training in CXL-based heterogeneous computing cluster,” in *Proc. IEEE INFOCOM Workshops*, 2024, pp. 1–6.
- [21] S.-S. Park *et al.*, “An LPDDR-based CXL-PNM platform for TCO-efficient inference of transformer-based large language models,” in *Proc. IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 970–982.
- [22] H. Liu *et al.*, “Enabling efficient large recommendation model training with near CXL memory processing,” in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024, pp. 382–395.
- [23] H. Zhang *et al.*, “SkyByte: Architecting an efficient memory-semantic CXL-based SSD with OS and hardware co-design,” in *Proc. IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 577–593.
- [24] H. A. Maruf *et al.*, “TPP: Transparent page placement for CXL-enabled tiered-memory,” in *Proc. ASPLOS*, 2023, pp. 742–755.
- [25] A. Raybuck *et al.*, “HeMem: Scalable tiered memory management for big data applications and real NVM,” in *Proc. ACM Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 392–407.
- [26] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, “DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022.
- [27] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proc. International Conference on Machine Learning (ICML)*, 2023.
- [28] Y. Lin *et al.*, “QServe: W4A8KV4 quantization and system co-design for efficient LLM serving,” in *Proc. Machine Learning and Systems (MLSys)*, 2025.
- [29] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.
- [30] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [31] B. Pang *et al.*, “Optimizing LLM inference throughput via memory-aware and SLA-constrained dynamic batching,” 2025.
- [32] Z. Liu *et al.*, “Deja vu: Contextual sparsity for efficient LLMs at inference time,” in *Proc. International Conference on Machine Learning (ICML)*, 2023, pp. 22 137–22 176.
- [33] C. Wild and J. Anderson, “Uncovering layer-dependent activation sparsity patterns in ReLU transformers,” 2024.
- [34] Y. Yang *et al.*, “CXLMemSim: A pure software simulated CXL.mem for performance characterization,” 2023.
- [35] Y. Wang *et al.*, “A full-system simulation framework for CXL-based SSD memory system,” 2025.